

II.2314 / II.2414 - Advanced Databases and Big Data

Big Data Project

Inclusive Mobility

Accessibility and Weather Risk Analysis

Authors: Mouzheng LI, Yuefan LIU

Date: 08/06/2026

Version: 1.0

[illegible]

Table of contents

Introduction.....	3
Data Sources	3
Project Structure.....	3
Pipeline Description	4
Data Ingestion.....	4
Data Formatting and Normalization.....	4
Data Combination and Aggregation	5
Indexing	6
Visualization	6
Technologies Used	7
Conclusion.....	7

Introduction

The goal of this project is to create a simple end-to-end data architecture, including data ingestion, data transformation, data indexing and data visualization. Our project focuses on inclusive mobility. We want to help a user know if it is a good idea to go out to a public place on a given day.

To do this, we combine accessibility data from French public establishments with weather forecast data. Accessibility alone is not enough, because a place can be accessible but the weather can make the trip difficult. Weather alone is also not enough, because a sunny day does not mean that the destination is suitable for people with reduced mobility.

Data Sources

Data is collected from two public APIs. The first one is AccesLibre, which gives accessibility information about establishments in France. This source contains useful fields such as wheelchair entrance, adapted toilets, disabled parking, flat entrance and entrance width.

The second source is Open-Meteo. It gives weather forecast information such as temperature, precipitation, wind speed and weather code. In our pipeline, the weather request is built from the cities found in AccesLibre, so the project is not limited to Paris only.

By using these two APIs together, the final data becomes more useful. The user can choose a city and a date, then see if the mobility conditions are good, risky, or need caution.

Project Structure

The project is organized with the same spirit as the course skeleton. The dags folder contains the Airflow workflow. The ingestion folder contains the API calls. The transform folder contains Spark jobs used to prepare and combine the data. The index folder is used to send the final usage data to Elasticsearch and to create the Kibana dashboard.

There are also utility modules for configuration, paths, S3 operations and Docker Spark execution. A kafka folder is present for the bonus streaming part, and the test folder contains unit tests for important parts of the pipeline.

```
inclusive-mobility-airflow/
+-- dags/                # Airflow DAG definition
+-- ingestion/           # AccesLibre and Open-Meteo API extraction
+-- transform/           # Spark formatting, scoring and verification jobs
+-- index/               # Elasticsearch import and Kibana dashboard setup
+-- kafka/               # Bonus weather streaming producer/consumer
+-- utils/               # Config, paths, S3 and Docker Spark helpers
+-- data/                # Local mirror of raw/formatted/usage data
+-- test/                # Unit tests
+-- reports/             # Project report drafts and image placeholders
+-- docker-compose.yaml
+-- .env.example
+-- README.md
+-- README_zh.md
```

Figure 1. Project structure and main components.

Pipeline Description

Our Big Data pipeline is orchestrated by Apache Airflow. The main DAG is called `inclusive_mobility_daily_pipeline`. It is scheduled daily and each task represents one step of the data flow.

The DAG starts by creating the S3 buckets in LocalStack. Then it extracts the AccesLibre data. After this step, it can extract the weather data because the weather task uses the city list from the accessibility records. Then Spark formats both datasets, combines them, computes scores, writes usage outputs, imports the results into Elasticsearch and finally prepares Kibana.

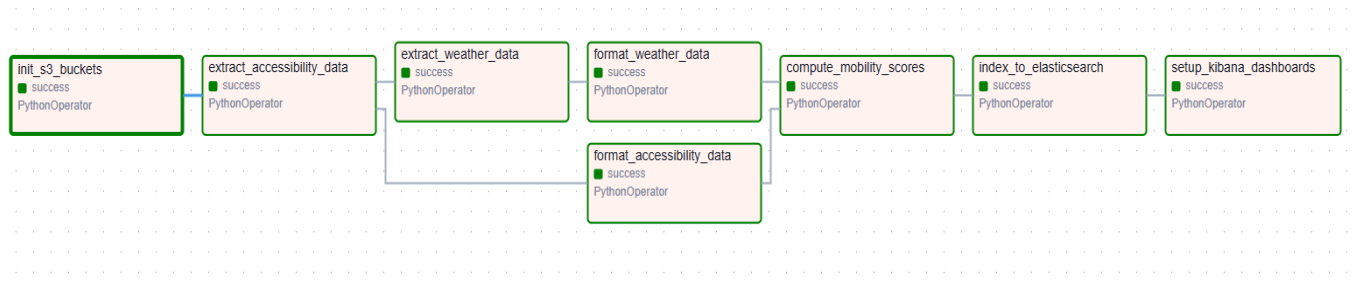


Figure 2. Airflow DAG graph reserved screenshot.

Data Ingestion

The ingestion part is written in Python. The AccesLibre script downloads establishment records and stores the raw JSON file. The Open-Meteo script reads the AccesLibre records, extracts unique cities and coordinates, and fetches daily weather forecasts for these locations.

All raw files are written into the raw layer of the data lake. We use LocalStack to simulate AWS S3 locally, so the project can run on a laptop while keeping a cloud-like architecture.

Data Formatting and Normalization

Raw JSON is not the best format for analysis. For this reason, we use Apache Spark to convert raw data into Parquet files in the formatted layer. This is more efficient and also gives us a cleaner schema.

For accessibility data, the Spark job renames fields and casts the values to useful types. For weather data, the Spark job flattens the daily arrays returned by Open-Meteo. The result is one row per city and weather date.

```

raw-data-mobility/
+-- acces_libre/
|   +-- establishments/
|       +-- {YYYYMMDD}/
|           +-- establishments.json
+-- open_meteo/
    +-- daily_weather/
        +-- {YYYYMMDD}/
            +-- daily_weather.json

formatted-data-mobility/
+-- acces_libre/
|   +-- establishments/
|       +-- {YYYYMMDD}/
|           +-- part-*.snappy.parquet
+-- open_meteo/
    +-- daily_weather/
        +-- {YYYYMMDD}/
            +-- part-*.snappy.parquet

usage-data-mobility/
+-- inclusive_mobility/
    +-- mobility_scores/
        |   +-- {YYYYMMDD}/
        |       +-- part-*.snappy.parquet
    +-- risky_areas/
        |   +-- {YYYYMMDD}/
        |       +-- part-*.snappy.parquet
    +-- improvement_priorities/
        |   +-- {YYYYMMDD}/
        |       +-- part-*.snappy.parquet
    +-- city_daily_summary/
        +-- {YYYYMMDD}/
            +-- part-*.snappy.parquet

```

Figure 3. Data lake layers and path convention.

Data Combination and Aggregation

Once both formatted datasets are ready, Spark joins accessibility rows with weather rows by city. This creates one enriched dataset where every establishment has accessibility information and weather information for the selected forecast dates.

The project computes three main scores. The `accessibility_score` is between 0 and 100, and a higher value is better. It is based on wheelchair access, adapted toilets, disabled parking, flat entrance and entrance width. The `weather_risk_score` is also between 0 and 100, but here a lower value is better, because it represents risk caused by precipitation, wind, temperature and severe weather codes.

The final mobility_score is calculated with the formula: $\text{mobility_score} = \text{accessibility_score} * 0.7 + (100 - \text{weather_risk_score}) * 0.3$. This means accessibility has the biggest influence, but bad weather can still reduce the final recommendation.

The usage layer contains several outputs: `mobility_scores` for detailed records, `risky_areas` for low mobility score places, `improvement_priorities` for places with low accessibility, and `city_daily_summary` for the dashboard recommendation cards.

Indexing

After Spark writes the usage outputs, the indexing step imports these Parquet datasets into Elasticsearch. The project creates indices for mobility scores, risky areas, improvement priorities and city daily summaries.

For the map, latitude and longitude are converted into a `geo_point` field called `location`. We also created score-class layers for the map, so low score places are red, medium score places are orange, and high score places are green.

Visualization

The final exposition layer is Kibana. The dashboard is named Should I Go Out Today? It is designed to be functional, not only decorative. A user can select a city and a weather date, then read the recommendation and inspect the places on the map.

The dashboard contains a recommendation card, average mobility score, number of risky places, number of safe places, a distribution chart, tables for safe and risky places, and a native Kibana map. The map uses the real latitude and longitude points and supports fit-to-data behavior.

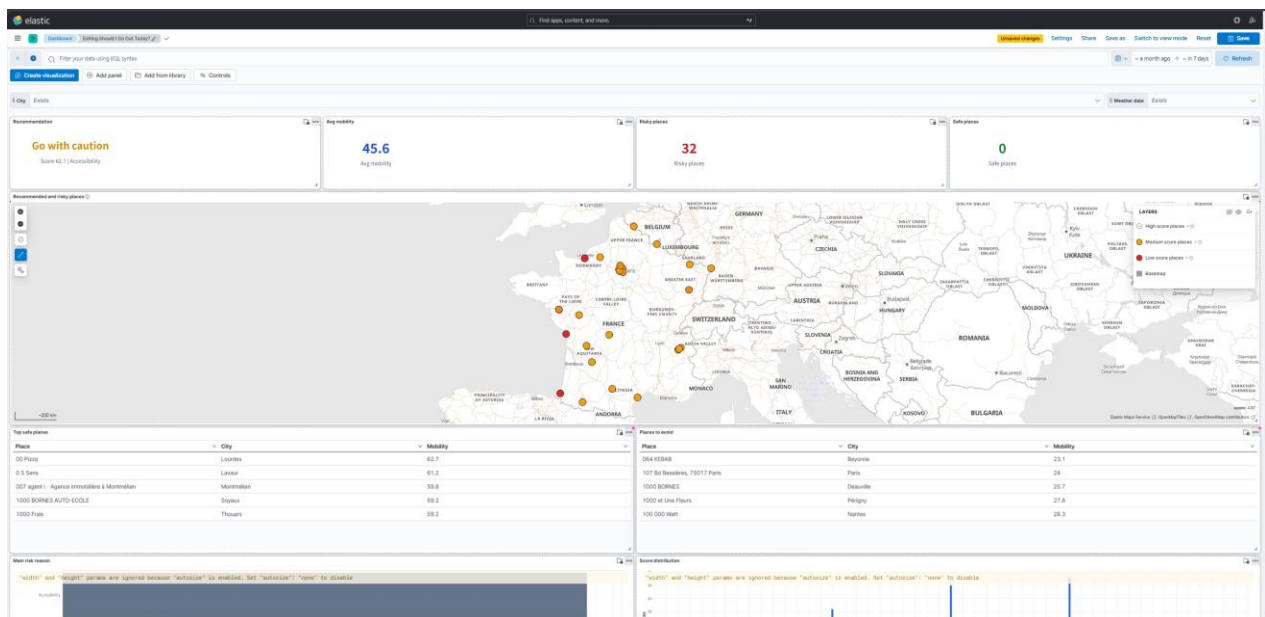


Figure 4. Kibana dashboard reserved screenshot.

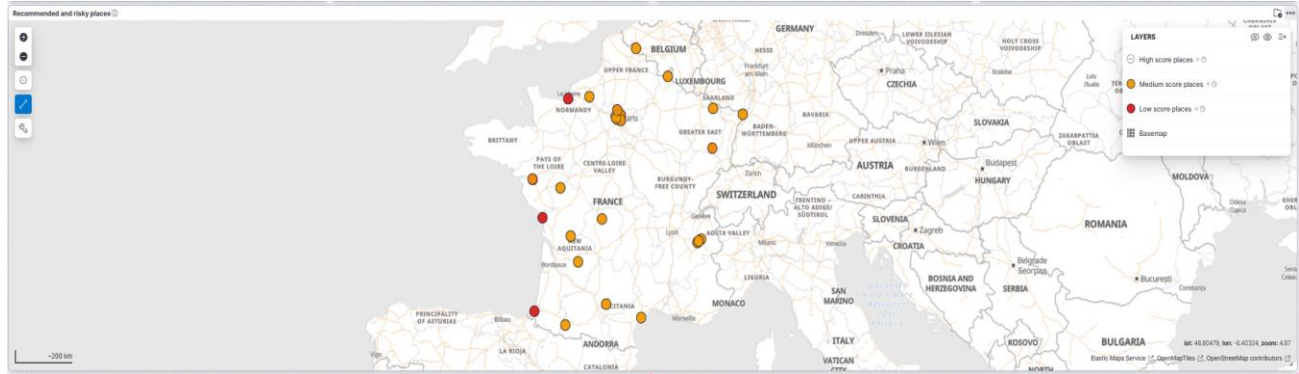


Figure 5. Kibana native map reserved screenshot.

Technologies Used

The project uses **Python** for API calls and scripts, **Apache Airflow** for orchestration, **Apache Spark** for distributed transformation, **LocalStack** to simulate **S3**, **Elasticsearch** for indexing, **Kibana** for visualization, **Kafka** for the bonus streaming component, and **Docker** to run the whole environment.

Using Docker makes the project easier to start and reproduce. The user can launch the services with docker compose, trigger the DAG in Airflow, and then open Kibana to explore the results.

Conclusion

This project demonstrates a complete Big Data pipeline from public API extraction to an interactive dashboard. It follows a data lake architecture with raw, formatted and usage layers, and it uses Airflow, Spark, Elasticsearch and Kibana to produce value from different data sources.

The result is a practical mobility analysis tool. It does not only show data; it helps answer a simple question: should I go out today, in this city, to this place? In the future, the project could include more locations, a more detailed accessibility model, more weather rules, and a real cloud S3 deployment instead of LocalStack.